

1 Results and Explanation

2 Evaluasi Latensi Inferensi pada ARM Cortex-M4F

Bagian ini menyajikan metodologi dan hasil evaluasi latensi inferensi model deteksi intrusi berbasis *tree ensemble* pada mikrokontroler ARM Cortex-M4F. Evaluasi ini menggunakan representasi *floating-point* 32-bit secara native dengan memanfaatkan *Floating-Point Unit* (FPU) perangkat keras yang tersedia pada arsitektur Cortex-M4F, sehingga tidak memerlukan kuantisasi. Model yang dievaluasi telah dilatih menggunakan teknik *oversampling* Conservative SMOTE tanpa *feature engineering*, di mana augmentasi sampel minoritas dibatasi hingga 20% dari jumlah kelas mayoritas untuk menghindari distorsi distribusi data pelatihan.

2.1 Justifikasi Pemilihan Platform ARM Cortex-M4F

Pemilihan mikrokontroler berbasis ARM Cortex-M4F sebagai target evaluasi didasarkan pada beberapa pertimbangan utama:

1. **Dukungan *Floating-Point* Perangkat Keras:** Arsitektur Cortex-M4F dilengkapi FPU yang mendukung operasi *single-precision* (32-bit) sesuai standar IEEE 754. Hal ini memungkinkan eksekusi model *machine learning* tanpa kuantisasi, sehingga menghilangkan risiko degradasi akurasi akibat konversi tipe data.
2. **Relevansi dengan Arsitektur LEACH:** Dalam protokol LEACH (*Low-Energy Adaptive Clustering Hierarchy*), setiap node sensor berpotensi menjadi *Cluster Head* (CH). Platform nRF52840, yang menggunakan prosesor Cortex-M4F, telah digunakan secara luas dalam implementasi WSN karena memiliki radio IEEE 802.15.4 terintegrasi yang kompatibel dengan protokol komunikasi sensor.
3. **Kapasitas Memori yang Memadai:** Dengan 1 MB *flash* dan 256 KB RAM, nRF52840 menyediakan ruang yang cukup untuk menyimpan model *tree ensemble* berskala besar tanpa memerlukan kompresi atau pembatasan yang agresif.
4. **Efisiensi Energi:** Arsitektur ARM Cortex-M4F dirancang untuk konsumsi daya rendah dengan mode *sleep* hingga $1,9 \mu A$, menjadikannya cocok untuk node sensor yang beroperasi dengan baterai terbatas.

2.2 Spesifikasi Platform Target

Table 1: Spesifikasi Mikrokontroler nRF52840 (ARM Cortex-M4F)

Parameter	Nilai
Arsitektur	32-bit ARM Cortex-M4F
Set Instruksi	ARMv7E-M + Thumb-2
<i>Floating-Point Unit</i>	FPv4-SP-D16 (single-precision)
Frekuensi Clock	64 MHz
Memori Flash	1 MB
Memori SRAM	256 KB
Radio Terintegrasi	IEEE 802.15.4, Bluetooth 5.0
Arus Aktif	4,8 mA @ 64 MHz (3,0V)
Mode Daya Rendah	1,9 μ A (System ON, RAM retention)
Tegangan Operasi	1,7V – 5,5V
Peripheral	Timer, UART, SPI, I2C, ADC, QSPI
Aplikasi Tipikal	Node sensor IoT/WSN, <i>edge computing</i>

2.3 Metodologi Evaluasi Latensi Inferensi

Metodologi evaluasi terdiri dari tujuh tahap utama: ekstraksi model, pra-pemrosesan *StandardScaler*, representasi data, optimasi kedalaman dan jumlah *stage*, evaluasi kualitas model, generasi kode C, serta kompilasi dan analisis siklus instruksi. Seluruh proses diimplementasikan dalam *pipeline* terotomasi menggunakan Python dan *cross-compiler* `arm-none-eabi-gcc`.

2.3.1 Tahap 1: Ekstraksi Model dari MLflow

Model *tree ensemble* yang dilatih menggunakan *scikit-learn* diekstrak dari artefak MLflow. Empat model dievaluasi, semuanya dilatih dengan teknik *oversampling* Conservative SMOTE tanpa *feature engineering*:

1. **Extra Trees** (ET): 100 *estimator*, kedalaman maksimum 20
2. **Random Forest** (RF): 100 *estimator*, kedalaman maksimum 20
3. **Gradient Boosting** (GB): 100 *estimator*, kedalaman maksimum 10, $\eta = 0,1$
4. **Histogram Gradient Boosting** (HistGB): 66 iterasi (dari `max_iter=100` dengan *early stopping*), kedalaman maksimum 15, $\eta = 0,1$

Setiap pohon keputusan diekstrak menggunakan metode *Breadth-First Search* (BFS) untuk menghasilkan representasi array datar (*flat array*) yang efisien untuk eksekusi pada mikrokontroler. Seluruh nilai *threshold* dipertahankan dalam format *float32* asli tanpa kuantisasi.

2.3.2 Tahap 2: Pra-pemrosesan StandardScaler

Selama pelatihan, *pipeline* MLflow menerapkan **StandardScaler** pada seluruh fitur sebelum melatih model. Transformasi ini menormalisasi setiap fitur x_j menjadi:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j} \quad (1)$$

di mana μ_j dan σ_j adalah rata-rata dan simpangan baku fitur j yang dihitung dari data pelatihan. Konsekuensinya, batas keputusan (*threshold*) seluruh node pohon berada dalam ruang fitur yang telah diskalakan, sehingga *StandardScaler* yang sama **harus** diterapkan pada data masukan saat inferensi.

Untuk deployment pada mikrokontroler, parameter *scaler* (μ_j dan σ_j untuk 16 fitur) disimpan sebagai array `const float32` di memori flash (128 byte tambahan). Overhead pra-pemrosesan sangat kecil:

$$C_{scaler} = N_{fitur} \times C_{scale} = 16 \times 5 = 80 \text{ siklus} \quad (2)$$

di mana $C_{scale} = 5$ siklus per fitur ($1 \times \text{VSUB.F32} + 1 \times \text{VDIV.F32} + \text{overhead akses memori}$). Ini mewakili kurang dari 1% dari total siklus inferensi.

2.3.3 Tahap 3: Representasi Data Tanpa Kuantisasi

Keuntungan utama penggunaan ARM Cortex-M4F adalah kemampuan untuk mempertahankan representasi *floating-point* asli dari model. Setiap node pohon keputusan direpresentasikan dalam format 12 byte yang sejajarkan (*aligned*):

Table 2: Format Node Pohon Keputusan Float32 pada ARM Cortex-M4F

Offset	Field	Ukuran	Keterangan
0–3	threshold	4 byte	<i>float32</i> , nilai batas / ID kelas
4–5	feature_idx	2 byte	<i>uint16</i> , 0xFFFF = node daun
6–7	left_child	2 byte	<i>uint16</i> , indeks anak kiri
8–9	right_child	2 byte	<i>uint16</i> , indeks anak kanan
10–11	padding	2 byte	penyejajaran ke 4 byte

Format 12 byte ini memberikan beberapa keunggulan dibanding format INT8 6 byte pada mikrokontroler 16-bit:

- **Tanpa *Error* Kuantisasi:** Tidak ada degradasi akurasi akibat konversi tipe data.
- **Tanpa Overhead Kuantisasi Fitur:** Operasi kuantisasi fitur pada saat inferensi tidak diperlukan, menghemat sekitar 650 siklus dibanding pendekatan INT8.
- **Akses Memori Tersejajarkan:** Ukuran 12 byte (kelipatan 4) memastikan akses memori yang efisien pada arsitektur 32-bit.
- **Perbandingan FPU Native:** Instruksi VCMPE.F32 melakukan perbandingan *floating-point* dalam 1 siklus clock.

2.3.4 Tahap 4: Optimasi Kedalaman Pohon dan Pencarian Konfigurasi

Meskipun nRF52840 memiliki 1 MB *flash*, model *tree ensemble* berukuran besar tetap memerlukan optimasi untuk menyisakan ruang bagi *protocol stack* dan kode aplikasi. Anggaran memori flash ditetapkan sebesar 800 KB ($\approx 78\%$ dari total), menyisakan ruang untuk:

- Stack protokol LEACH dan MAC 802.15.4 ($\approx 50\text{--}80$ KB)
- Driver radio dan HAL (≈ 40 KB)
- Kode aplikasi sensor dan *buffer* ($\approx 30\text{--}50$ KB)

Strategi Pencarian Pencarian konfigurasi optimal dibedakan berdasarkan jenis model:

- **Model *bagging*** (Random Forest, Extra Trees): Seluruh 100 pohon dipertahankan; hanya kedalaman yang divariasikan dari himpunan $D = \{6, 8, 10, 12, 15, 20, \infty\}$ (∞ berarti tanpa batas).
- **Model *boosting*** (GB, HistGB): Kedalaman **dan** jumlah *stage* divariasikan, dengan $D = \{6, 8, 10, 12, 15, 20, \infty\}$ dan $S = \{10, 20, 50, N_{max}\}$, karena model *boosting* multi-kelas menghasilkan $N_{stage} \times N_{kelas}$ pohon.

Kriteria pemilihan konfigurasi optimal menggunakan prinsip *fidelity* tertinggi:

$$\text{config}^* = \arg \min_{c \in \mathcal{C}_{fit}} |\Delta F1_c| \quad \text{di mana} \quad \mathcal{C}_{fit} = \{c : M_c \leq 800 \text{ KB}\} \quad (3)$$

dengan $\Delta F1_c = F1_{ARM,c} - F1_{original}$ dan M_c adalah ukuran memori konfigurasi c . Konfigurasi yang menghasilkan $\Delta F1 \geq -0,01$ dinyatakan memenuhi syarat kualitas.

Hasil Pencarian Konfigurasi Tabel 3 menunjukkan konfigurasi optimal terpilih untuk setiap model:

Table 3: Konfigurasi Optimal Terpilih untuk Setiap Model

Model	Kedalaman	Stages	Pohon	Node Total*	Memori (KB)
Extra Trees	10	–	100	49.504	581,1
Random Forest	10	–	100	52.710	618,6
Gradient Boosting	10	10	50	48.670	570,9
HistGrad. Boosting	15	10	50	3.050	36,3

*Jumlah total node tersimpan di seluruh pohon; bukan jumlah node yang ditraversal saat inferensi.

Tabel 4 menunjukkan eksplorasi pencarian yang lebih rinci untuk mengilustrasikan *trade-off* antara ukuran memori dan kualitas model:

Temuan penting dari pencarian konfigurasi:

1. **Extra Trees:** Model ini menunjukkan degradasi kualitas yang paling parah pada setiap kedalaman yang memenuhi batasan memori. Bahkan pada kedalaman 10 (konfigurasi terbesar yang muat, 581 KB), $\Delta F1_M = -0,140$. Kualitas hanya membaik pada kedalaman 15+ yang memerlukan 2,8 MB — jauh melebihi kapasitas flash. Hal ini menunjukkan bahwa sifat pemilihan *split* acak pada Extra Trees memerlukan pohon yang sangat dalam untuk mempartisi ruang fitur secara efektif.
2. **Random Forest:** Kualitas model menurun secara monoton dengan berkurangnya kedalaman, namun degradasinya lebih moderat ($\Delta F1_M = -0,027$ pada kedalaman 10). Kedalaman 15 memberikan $\Delta F1_M = -0,005$ (sangat baik), namun memerlukan 2,5 MB. Pada kedalaman 10 yang merupakan konfigurasi terbesar yang muat, penurunan terutama terlihat pada kelas minoritas Grayhole ($\Delta F1 = -0,114$).
3. **Gradient Boosting:** Transisi kualitas yang sangat tajam terjadi antara kedalaman 8 ($\Delta F1_M = -0,631$) dan 10 ($\Delta F1_M = -0,002$). Hal

Table 4: Eksplorasi Pencarian Konfigurasi: Pengaruh Kedalaman dan Jumlah *Stage* terhadap Kualitas

Model	Kedalaman	<i>St.</i>	Node Total	Memori	$\Delta F1_M$	Muat?
Extra Trees	6	–	8.216	97,2 KB	–0,165	Ya
	8	–	21.654	254,7 KB	–0,146	Ya
	10	–	49.504	581,1 KB	–0,140	Ya
	15	–	235.494	2.761 KB	–0,107	Tidak
Random Forest	6	–	8.796	104,0 KB	–0,066	Ya
	10	–	52.710	618,6 KB	–0,027	Ya
	12	–	102.334	1.200 KB	–0,021	Tidak
	15	–	217.154	2.546 KB	–0,005	Tidak
Grad. Boosting	6	10	5.534	65,4 KB	–0,680	Ya
	8	10	17.612	206,9 KB	–0,631	Ya
	10	10	48.670	570,9 KB	–0,002	Ya
	10	20	110.166	1.292 KB	+0,003	Tidak
HistGrad. Boosting	6	10	1.226	14,9 KB	–0,056	Ya
	10	10	2.438	29,1 KB	–0,021	Ya
	15	10	3.050	36,3 KB	+0,001	Ya
	15	66	20.130	238,6 KB	+0,002	Ya

ini disebabkan karena pohon GB secara alami memiliki kedalaman efektif ≤ 10 — pembatasan ke kedalaman 10 **tidak melakukan pemotongan** sama sekali. Kualitas yang sangat baik dicapai hanya dengan 10 dari 100 *stage* boosting, menunjukkan bahwa iterasi awal sudah menangkap pola utama dalam data.

4. **HistGradient Boosting:** Model ini mencapai konfigurasi paling ringkas (36,3 KB) dengan $\Delta F1_M = +0,001$ — kualitas justru sedikit *meningkat* dibanding model asli. Pohon HistGB secara alami memiliki kedalaman ≤ 15 , sehingga tidak ada pemotongan. Hanya 10 dari 66 iterasi diperlukan. Peningkatan kualitas mengindikasikan bahwa iterasi boosting selanjutnya menyebabkan sedikit *overfitting* pada data pelatihan.

2.3.5 Tahap 5: Evaluasi Kualitas Model

Evaluasi kualitas dilakukan untuk memastikan bahwa optimasi kedalaman dan pengurangan *stage* tidak menyebabkan degradasi performa yang tidak dapat diterima. Evaluasi menggunakan *test set* yang sama seperti pada pelatihan (20% data, *stratified split*, `random.state=42`).

Metodologi Evaluasi Kualitas Untuk setiap model, dibandingkan prediksi asli (*scikit-learn*) dengan prediksi model yang telah dioptimasi menggunakan implementasi traversal pohon Python yang identik secara fungsional dengan implementasi C:

1. Muat model asli dari artefak MLflow
2. Muat dan terapkan *StandardScaler* yang sama pada data uji
3. Hasilkan prediksi asli menggunakan `model.predict(X_test)`
4. Hitung metrik asli: akurasi, $F1_{macro}$, $F1_{weighted}$, serta $F1$ per kelas
5. Ekstrak pohon dengan pembatasan kedalaman dan/atau *stage*
6. Lakukan traversal pohon secara manual dan agregasi (*voting* atau *scoring*)
7. Hitung metrik yang sama dan bandingkan ($\Delta F1_{macro} \geq -0,01$ sebagai syarat kualitas)

Table 5: Perbandingan Kualitas Model: Asli vs. ARM-Optimized (Conservative SMOTE)

Model	Akurasi Asli	Akurasi ARM	$F1_M$ Asli	$F1_M$ ARM	Status
Extra Trees	0,9908	0,9590	0,9496	0,8096	DROP ($-0,140$)
Random Forest	0,9957	0,9855	0,9741	0,9447	DROP ($-0,029$)
Grad. Boosting	0,9964	0,9947	0,9763	0,9712	OK ($-0,005$)
HistGrad. Boosting	0,9944	0,9946	0,9658	0,9687	OK ($+0,003$)

Hasil Evaluasi Kualitas Berbeda dengan model SMOTE-ENN yang menunjukkan akurasi asli rendah ($\approx 48-83\%$), model Conservative SMOTE mencapai akurasi asli di atas 99% untuk seluruh empat model, mengkonfirmasi bahwa strategi Conservative SMOTE tidak mendistorsi proses pelatihan.

Hasil evaluasi per kelas ditunjukkan pada Tabel 6:

Analisis Hasil Kualitas

1. **Extra Trees:** Model ini menunjukkan degradasi signifikan ($\Delta F1_M = -0,140$) yang **tidak dapat diterima**. Kelas Grayhole mengalami penurunan paling drastis ($F1 : 0,921 \rightarrow 0,580$, $\Delta = -0,341$). Hal ini

Table 6: Skor $F1$ per Kelas: Asli vs. ARM-Optimized

Model	Metrik	Blackhole	Flooding	Grayhole	Normal	TDMA
Extra Trees	Asli	0,9525	0,9348	0,9210	0,9965	0,9432
	ARM	0,7841	0,8031	0,5801	0,9858	0,8950
Random Forest	Asli	0,9911	0,9532	0,9786	0,9979	0,9498
	ARM	0,9769	0,9474	0,8646	0,9927	0,9418
Grad. Boosting	Asli	0,9913	0,9585	0,9874	0,9983	0,9459
	ARM	0,9823	0,9580	0,9633	0,9976	0,9547
HistGrad. Boosting	Asli	0,9879	0,9519	0,9788	0,9973	0,9131
	ARM	0,9806	0,9496	0,9621	0,9976	0,9538

disebabkan oleh sifat pemilihan *split* acak Extra Trees yang menghasilkan pohon sangat dalam (rata-rata kedalaman asli ≈ 49): pemotongan ke kedalaman 10 menghilangkan sebagian besar kapasitas diskriminatif. Bahkan pada kedalaman 20 ($\Delta F1_M = -0,054$), model masih belum memenuhi ambang batas $-0,01$.

2. **Random Forest:** Degradasi lebih moderat ($\Delta F1_M = -0,029$) namun masih melebihi ambang batas $-0,01$. Penurunan terkonsentrasi pada kelas Grayhole ($\Delta F1 = -0,114$), sementara kelas lain relatif terjaga. Random Forest memerlukan kedalaman 15 ($\Delta F1_M = -0,005$, 2,5 MB) untuk memenuhi syarat — terlalu besar untuk batasan 800 KB.
3. **Gradient Boosting:** Kualitas dipertahankan sangat baik ($\Delta F1_M = -0,005$) hanya dengan 10 dari 100 *stage* boosting pada kedalaman alami 10. Selisih terbesar pada kelas Grayhole ($\Delta F1 = -0,024$) masih dalam toleransi. Kelas TDMA bahkan menunjukkan sedikit peningkatan ($\Delta F1 = +0,009$).
4. **Histogram Gradient Boosting:** Kualitas *meningkat* ($\Delta F1_M = +0,003$). Peningkatan paling signifikan pada kelas TDMA ($F1 : 0,913 \rightarrow 0,954$, $\Delta = +0,041$), mengindikasikan bahwa pengurangan dari 66 ke 10 iterasi mengurangi *overfitting*. Kelas Grayhole mengalami penurunan kecil ($\Delta F1 = -0,017$) namun dikompensasi oleh peningkatan pada kelas lain.

Berdasarkan kriteria $\Delta F1_M \geq -0,01$, **dua dari empat model** (Gradient Boosting dan Histogram Gradient Boosting) memenuhi syarat kualitas untuk deployment. Extra Trees **tidak layak** karena pemotongan kedalaman yang terlalu agresif, dan Random Forest berada di batas kelayakan

$(\Delta F1_M = -0,029)$.

2.3.6 Tahap 6: Generasi Kode C untuk ARM Cortex-M4F

Kode C yang dihasilkan disesuaikan dengan arsitektur ARM Cortex-M4F untuk memanfaatkan FPU dan instruksi Thumb-2 secara optimal. Untuk setiap model, dihasilkan tujuh file:

1. `model_config.h`: Definisi konfigurasi model (jumlah pohon, fitur, kelas)
2. `tree_data.h`: Data pohon dalam format array `const` (disimpan di flash)
3. `inference.h`: Deklarasi fungsi inferensi
4. `inference.c`: Implementasi fungsi traversal pohon dan agregasi ensemble
5. `main.c`: Program *benchmark* menggunakan *DWT Cycle Counter*
6. `Makefile`: Konfigurasi kompilasi untuk ARM Cortex-M4F
7. `linker.ld`: *Linker script* minimal untuk nRF52840

Dua Varian Inferensi Terdapat dua varian implementasi inferensi yang mencerminkan perbedaan mekanisme agregasi:

1. **Voting** (RF, ET): Setiap pohon menghasilkan prediksi kelas, kemudian kelas dengan suara terbanyak dipilih sebagai hasil akhir.
2. **Scoring** (GB, HistGB): Setiap pohon menghasilkan skor *floating-point* yang dijumlahkan per kelas. Pada GB, skor dikalikan *learning rate* ($\eta = 0,1$); pada HistGB, *learning rate* sudah terintegrasi ke dalam nilai daun saat pelatihan. Kelas dengan skor tertinggi dipilih.

Pseudocode untuk traversal pohon dan inferensi *ensemble* ditunjukkan pada Algoritma 1 dan 2.

Algorithm 1 Traversal Pohon Keputusan Float32 pada ARM Cortex-M4F

Require: nodes: array TreeNode, features: array float32[16]

Ensure: predicted_class (voting) atau leaf_value (scoring)

```
1: node  $\leftarrow$  nodes[0]
2: while node.feature_idx  $\neq$  0xFFFF do
3:   if features[node.feature_idx]  $\leq$  node.threshold then
4:     node  $\leftarrow$  nodes[node.left_child] {VCMPE.F32 + ITE}
5:   else
6:     node  $\leftarrow$  nodes[node.right_child]
7:   end if
8: end while
9: return (uint8_t) node.threshold {ID kelas atau skor daun}
```

Algorithm 2 Inferensi Ensemble (Varian Voting dan Scoring)

Require: features: float32[16], trees: array of tree pointers

Ensure: predicted_class: uint8_t

```
1: if mode = VOTING then
2:   votes[N_CLASSES]  $\leftarrow$  {0}
3:   for  $i \leftarrow 0$  to N_TREES - 1 do
4:     cls  $\leftarrow$  tree_predict(trees[i], features)
5:     votes[cls]++
6:   end for
7:   return  $\arg \max_c$  votes[c]
8: else
9:   {SCORING}
10:  scores[N_CLASSES]  $\leftarrow$  initial_scores
11:  for  $i \leftarrow 0$  to N_TREES - 1 do
12:    leaf  $\leftarrow$  tree_predict_leaf(trees[i], features)
13:    cls  $\leftarrow$  imod N_CLASSES
14:    scores[cls]  $\leftarrow$  scores[cls] +  $\eta \times$  leaf
15:  end for
16:  return  $\arg \max_c$  scores[c]
17: end if
```

2.3.7 Tahap 7: Kompilasi dan Analisis Siklus

Kompilasi untuk ARM Cortex-M4F Kode C dikompilasi menggunakan `arm-none-eabi-gcc` versi 15.2.0 dengan opsi yang mengaktifkan FPU perangkat keras:

```
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -mfpv4-sp-d16 \
    -mfloat-abi=hard -O2 -ffreestanding -nostdlib -nostartfiles \
    -T linker.ld -o inference.elf main.o inference.o -lgcc
```

Opsi kompilasi yang digunakan:

- `-mcpu=cortex-m4`: Target prosesor Cortex-M4
- `-mthumb`: Gunakan set instruksi Thumb-2 (16/32-bit campuran)
- `-mfpv4-sp-d16`: Aktifkan FPU *single-precision* dengan 16 register *double-word*
- `-mfloat-abi=hard`: Gunakan konvensi pemanggilan *hardware float* (argumen float melalui register FPU)
- `-O2`: Optimasi agresif termasuk *inlining* dan *loop optimization*

Analisis Disassembly dan Estimasi Siklus Estimasi siklus diperoleh melalui analisis statis (*static cycle counting*) terhadap *disassembly* biner yang dikompilasi, bukan melalui perhitungan teoritis semata. Proses analisis berlangsung dalam empat langkah:

1. **Kompilasi dan *disassembly***: Kode C dikompilasi dengan opsi yang telah disebutkan, kemudian file ELF diuraikan menggunakan perintah:

```
arm-none-eabi-objdump -d inference_benchmark.elf
```

2. **Identifikasi *inner loop***: Dari keluaran *objdump*, fungsi `ensemble_predict` dianalisis untuk menemukan *inner loop* traversal pohon. Kompilator dengan opsi `-O2` melakukan *inlining* fungsi `tree_predict` ke dalam `ensemble_predict`, sehingga *loop* traversal muncul langsung dalam badan fungsi `ensemble_predict`. Berikut adalah *inner loop* traversal yang dihasilkan oleh kompilator (diekstrak dari *disassembly* aktual model Histogram Gradient Boosting):

```

e6:  eb00 0282  add.w  r2, r0, r2, lsl #2
ea:  ed92 7a00  vldr   s14, [r2]
ee:  eef4 7ac7  vcmpe.f32  s15, s14
f2:  eef1 fa10  vmrs   APSR_nzcv, fpscr
f6:  bfac      ite    ge
f8:  88db      ldrhge r3, [r3, #6]
fa:  891b      ldrhlt r3, [r3, #8]
fc:  eb03 0343  add.w  r3, r3, r3, lsl #1
100: eb01 0383  add.w  r3, r1, r3, lsl #2
104: edd3 7a00  vldr   s15, [r3]
108: 889a      ldrh   r2, [r3, #4]
10a: 4562      cmp    r2, ip
10c: d1eb      bne.n  e6

```

3. **Pemetaan instruksi ke siklus:** Setiap instruksi dalam *loop* dipetakan ke jumlah siklus berdasarkan *ARM Cortex-M4 Technical Reference Manual* (ARM DDI 0439D), yang mendokumentasikan latensi setiap instruksi pada *pipeline* tiga tahap Cortex-M4:

Table 7: Pemetaan Instruksi *Inner Loop* ke Siklus ARM Cortex-M4

Alamat	Instruksi	Operasi	Referensi TRM	Siklus
0xe6	ADD.W	Alamat fitur: $r_0 + \text{idx} \times 4$	Data processing, 1c	1
0xea	VLDR	Muat fitur dari memori	FP load, 2c	2
0xee	VCMPE.F32	Perbandingan FPU	FP compare, 1c	1
0xf2	VMRS	Transfer flag FPU $\rightarrow$ APSR	FP status, 1c	1
0xf6	ITE GE	Blok kondisional <i>If-Then-Else</i>	IT block, 0c	0
0xf8	LDRH (cond.)	Muat <code>left</code> atau <code>right</code> child	Load half, 2c	2
0xfc	ADD.W	$\text{idx} \times 3$ ($\text{idx} + \text{idx} \ll 1$)	Data processing, 1c	1
0x100	ADD.W	Alamat node: $\text{base} + \text{off} \times 4$	Data processing, 1c	1
0x104	VLDR	Muat <i>threshold</i> node baru	FP load, 2c	2
0x108	LDRH	Muat <code>feature_idx</code>	Load half, 2c	2
0x10a	CMP	Periksa node daun (0xFFFF)	Data processing, 1c	1
0x10c	BNE	Kembali ke awal <i>loop</i>	Branch, 1+Pc	1
Total per iterasi (satu node)				15

Catatan: Instruksi ITE (*If-Then-Else*) pada arsitektur Thumb-2 memerlukan 0 siklus tambahan — ia hanya mengkondisikan eksekusi instruksi berikutnya. Instruksi BNE diasumsikan 1 siklus karena *branch prediction* pada jalur *loop-back* yang dominan; pada kasus *misprediction*

(keluar *loop*), terjadi *pipeline refill* 1–3 siklus tambahan, namun ini hanya terjadi satu kali per pohon. Instruksi VLDR dan LDRH masing-masing memerlukan 2 siklus sesuai tabel latensi instruksi pada ARM Cortex-M4 TRM (ARM DDI 0439D), yang memperhitungkan 1 siklus eksekusi + 1 siklus akses bus memori.

4. **Analisis komponen non-traversal:** Selain *inner loop*, komponen lain juga diidentifikasi dari *disassembly* yang sama:

- **Overhead per pohon** (10 siklus): Teridentifikasi dari instruksi di luar *loop* yang menginisialisasi pointer pohon, memuat alamat array node, dan memeriksa node awal. Terdiri dari: LDR (muat pointer pohon, 1c) + MOV (salin pointer, 1c) + LDRH (muat *feature_idx* awal, 2c) + VLDR (muat *threshold* awal, 2c) + CMP (cek node daun awal, 1c) + overhead transisi antar-pohon (3c).
- **Agregasi *voting*** (3 siklus/pohon): Setelah traversal, hasil kelas diakumulasi. Dari *disassembly* model Random Forest:

```

106:  eefc 7ae7  vcvtt.u32.f32  s15, s15
10e:  f89d 3004  ldrb.w  r3, [sp, #4]
116:  eb0d 0343  add.w  r3, sp, r3, lsl #1
11a:  899a      ldrh  r2, [r3, #12]
11c:  3201      adds  r2, #1
11e:  819a      strh  r2, [r3, #12]

```

Inti votasi: LDRH (muat counter, 2c) + ADDS (increment, 1c) + STRH (simpan, 1c) = 4 siklus/pohon. Instruksi konversi dan kalkulasi alamat dihitung sebagai overhead per-pohon.

- **Agregasi *scoring*** (5 siklus/pohon): Dari *disassembly* model HistGradient Boosting:

```

10e:  f814 3f01  ldrb.w  r3, [r4, #1]!
114:  eb02 0383  add.w  r3, r2, r3, lsl #2
118:  ed13 7a05  vldr  s14, [r3, #-20]
11c:  ee77 7a27  vadd.f32  s15, s14, s15
122:  ed43 7a05  vstr  s15, [r3, #-20]

```

Inti akumulasi skor: LDRB (muat indeks kelas, 1c) + VLDR (muat skor lama, 2c) + VADD.F32 (tambah skor, 1c) + VSTR (simpan skor baru, 1c) + overhead alamat (1c) = 6 siklus/pohon. Pada model HistGB, *learning rate* sudah terintegrasi dalam nilai daun, sehingga tidak ada operasi perkalian tambahan.

- **Argmax voting** (15 siklus): Dari *disassembly* Random Forest, loop *argmax* iterasi 5 kelas:

```

13a:  f834 1b02  ldrh.w  r1, [r4], #2
13e:  fa5f fc83  uxtb.w  ip, r3
142:  3301      adds    r3, #1
144:  4291      cmp     r1, r2
146:  bf88      it     hi
148:  4660      movhi   r0, ip
14a:  4291      cmp     r1, r2
14c:  bf88      it     hi
14e:  460a      movhi   r2, r1
150:  2b05      cmp     r3, #5
152:  d1f2      bne.n   13a

```

Sekitar $4 \text{ siklus/kelas} \times 5 \text{ kelas} = 20 \text{ siklus}$ (LDRH.W memerlukan 2 siklus; beberapa instruksi di-*fold* oleh blok IT).

- **Argmax scoring** (30 siklus): Dari *disassembly* HistGB, termasuk inisialisasi skor awal dan loop *argmax* FPU:

```

136:  eddd 7a01  vldr     s15, [sp, #4]
13a:  2300      movs    r3, #1
13e:  2000      movs    r0, #0
140:  ecb2 7a01  vldmia   r2!, {s14}
144:  eeb4 7ae7  vcmpe.f32 s14, s15
148:  eef1 fa10  vmrs     APSR_nzcv, fpscr
14c:  b2d9      uxtb    r1, r3
14e:  f103 0301  add.w    r3, r3, #1
152:  bfc8      it     gt
154:  4608      movgt    r0, r1
156:  bfc8      it     gt
158:  eef0 7a47  vmovgt.f32 s15, s14
15c:  2b05      cmp     r3, #5
15e:  d1ef      bne.n   140

```

Inisialisasi (4c) + 5 iterasi $\times \approx 6$ siklus/iterasi (VLDMIA 2c + VCMPE + VMRS + CMP + BNE, beberapa tanpa biaya karena blok IT) ≈ 35 siklus.

Dengan demikian, seluruh komponen siklus diperoleh dari analisis instruksi aktual pada biner yang dikompilasi, bukan dari estimasi teoritis. Jumlah siklus per node pada *inner loop* adalah **15 siklus**, diperoleh dari

pemetaan langsung setiap instruksi ke latensi siklus berdasarkan *ARM Cortex-M4 Technical Reference Manual* (ARM DDI 0439D). Nilai ini sudah memperhitungkan latensi akses memori 2 siklus untuk instruksi *load* (VLDR, LDRH), sehingga tidak diperlukan margin tambahan.

Model Estimasi Siklus Total Total siklus inferensi dihitung berdasarkan:

$$C_{total} = C_{scaler} + N_{pohon} \times (C_{overhead} + d_{rata-rata} \times C_{per_node} + C_{agg/pohon}) + C_{argmax} \quad (4)$$

di mana seluruh nilai parameter diperoleh dari analisis *disassembly*:

- C_{scaler} = 80 siklus (pra-pemrosesan *StandardScaler*, Subbagian 2.3.2)
- C_{per_node} = 15 siklus (dari analisis *disassembly*, Tabel 7)
- $C_{overhead}$ = 10 siklus per pohon (dari instruksi inisialisasi di luar *loop*)
- $d_{rata-rata}$ = kedalaman rata-rata traversal pohon (dihitung dari struktur pohon yang diekstrak)

Untuk komponen agregasi (dari *disassembly*):

- **Voting** (ET, RF): $C_{agg/pohon} = 4$ siklus (LDRH 2c + ADDS + STRH), $C_{argmax} = 20$ siklus
- **Scoring** (GB, HistGB): $C_{agg/pohon} = 6$ siklus (LDRB + VLDR 2c + VADD.F32 + VSTR + calc), $C_{argmax} = 35$ siklus

Perlu dicatat bahwa jumlah node yang ditraversal saat inferensi jauh lebih kecil dari total node yang tersimpan di memori *flash* (Tabel 3). Untuk setiap pohon, hanya satu jalur dari akar ke daun yang dilalui, dengan panjang rata-rata $d_{rata-rata}$ node. Sebagai contoh, Random Forest memiliki 52.710 node total di seluruh 100 pohon, namun saat inferensi hanya $100 \times 10 = 1.000$ node yang ditraversal (masing-masing memerlukan 15 siklus, menghasilkan 15.000 siklus traversal). Total node menentukan penggunaan memori *flash*, sedangkan kedalaman rata-rata menentukan jumlah siklus inferensi.

2.4 Hasil Evaluasi Latensi Inferensi

2.4.1 Penggunaan Memori

Penggunaan memori flash menunjukkan variasi yang sangat besar. HistGradient Boosting memanfaatkan hanya 3,6% dari kapasitas flash (37,6 KB),

Table 8: Penggunaan Memori pada nRF52840 (1 MB Flash)

Model	Pohon	Node Total	.text (byte)	Total (byte)	Flash (%)
Extra Trees [†]	100	49.504	595.120	595.148	56,8%
Random Forest [†]	100	52.710	633.592	633.620	60,4%
Gradient Boosting	50	48.670	584.996	585.024	55,8%
HistGrad. Boosting	50	3.050	37.552	37.580	3,6%

[†]Tidak memenuhi syarat kualitas ($\Delta F1_M < -0,01$).

sementara tiga model lainnya menggunakan 55–61%. Meskipun keempat model memenuhi batasan 800 KB, perbedaan ini mencerminkan efisiensi representasi model *boosting* berbasis histogram:

- **Model *bagging*** (ET, RF) mempertahankan seluruh 100 pohon independen dengan kedalaman 10, menghasilkan ≈ 50.000 node.
- **GB** hanya menggunakan 10 dari 100 *stage* (50 pohon = 10 *stage* $\times$ 5 kelas), namun setiap pohon memiliki kedalaman penuh 10 dengan rata-rata 973 node/pohon.
- **HistGB** menggunakan 10 dari 66 iterasi (50 pohon) dengan rata-rata hanya 61 node/pohon, mencerminkan representasi histogram yang jauh lebih ringkas.

2.4.2 Rincian Jumlah Siklus

Table 9: Rincian Jumlah Siklus per Komponen pada ARM Cortex-M4F

Komponen	Extra Trees [†]	Random Forest [†]	Grad. Boosting	HistGrad. Boosting
<i>StandardScaler</i>	80	80	80	80
Traversal Pohon ($\times N$)	15.000	15.000	7.500	9.875
Overhead per Pohon ($\times N$)	1.000	1.000	500	500
Agregasi per Pohon ($\times N$)	400	400	300	300
Agregasi <i>Argmax</i>	20	20	35	35
Total Siklus (rata-rata)	16.500	16.500	8.415	10.790

[†]Tidak memenuhi syarat kualitas.

Catatan mengenai rincian siklus:

- Komponen *StandardScaler* (80 siklus) mewakili kurang dari 1% dari total siklus inferensi.

- HistGradient Boosting memiliki traversal sedikit lebih tinggi per pohon (rata-rata kedalaman 13,18 vs. 10) namun menghasilkan total yang lebih rendah dari model *bagging* karena jumlah pohon yang jauh lebih sedikit (50 vs. 100).
- Overhead agregasi *scoring* (GB, HistGB) sedikit lebih tinggi karena melibatkan operasi *floating-point* (perkalian *learning rate* dan penjumlahan skor).

Contoh Perhitungan Rinci Berikut disajikan perhitungan rinci untuk setiap komponen siklus pada masing-masing model, menggunakan formula:

$$C_{total} = C_{scaler} + N_{pohon} \times (C_{overhead} + d_{rata-rata} \times C_{per_node} + C_{agg/pohon}) + C_{argmax} \quad (5)$$

1. Extra Trees (100 pohon, kedalaman rata-rata traversal = 10, agregasi *voting*):

$$C_{scaler} = 16 \times 5 = 80 \text{ siklus}$$

$$C_{traversal} = N_{pohon} \times d_{rata-rata} \times C_{per_node} = 100 \times 10 \times 15 = 15.000 \text{ siklus}$$

$$C_{overhead} = N_{pohon} \times 10 = 100 \times 10 = 1.000 \text{ siklus}$$

$$C_{agregasi} = N_{pohon} \times C_{agg/pohon} = 100 \times 4 = 400 \text{ siklus}$$

$$C_{argmax} = 20 \text{ siklus (voting: 5 kelas)}$$

$$C_{total} = 80 + 15.000 + 1.000 + 400 + 20 = \mathbf{16.500} \text{ siklus}$$

2. Random Forest (100 pohon, kedalaman rata-rata traversal = 10, agregasi *voting*):

Perhitungan identik dengan Extra Trees karena memiliki parameter yang sama (100 pohon, kedalaman terbatas 10, agregasi *voting*):

$$C_{total} = 80 + (100 \times 10 \times 15) + (100 \times 10) + (100 \times 4) + 20 = \mathbf{16.500} \text{ siklus}$$

Meskipun RF memiliki lebih banyak node total tersimpan (52.710 vs. 49.504 pada ET), jumlah siklus inferensi identik karena kedalaman rata-rata traversal sama ($d = 10$). Node tambahan berada pada cabang yang tidak dilalui untuk sampel tertentu, dan hanya memengaruhi penggunaan memori *flash*.

3. Gradient Boosting (50 pohon = 10 *stage* $\times$ 5 kelas, kedalaman

rata-rata traversal = 10, agregasi *scoring*):

$$\begin{aligned}
C_{scaler} &= 16 \times 5 = 80 \text{ siklus} \\
C_{traversal} &= N_{pohon} \times d_{rata-rata} \times C_{per_node} = 50 \times 10 \times 15 = 7.500 \text{ siklus} \\
C_{overhead} &= N_{pohon} \times 10 = 50 \times 10 = 500 \text{ siklus} \\
C_{agregasi} &= N_{pohon} \times C_{agg/pohon} = 50 \times 6 = 300 \text{ siklus} \\
C_{argmax} &= 35 \text{ siklus (scoring: inisialisasi + argmax 5 kelas)} \\
C_{total} &= 80 + 7.500 + 500 + 300 + 35 = \mathbf{8.415} \text{ siklus}
\end{aligned}$$

Jumlah pohon yang lebih sedikit (50 vs. 100) dan mekanisme agregasi *scoring* menghasilkan total siklus yang jauh lebih rendah dibanding model *bagging*. Meskipun $C_{agg/pohon}$ *scoring* (6 siklus) lebih tinggi dari *voting* (4 siklus), penghematan dari pengurangan jumlah pohon lebih dominan.

4. Histogram Gradient Boosting (50 pohon = 10 iterasi $\times$ 5 kelas, kedalaman rata-rata traversal $\approx 13,17$, agregasi *scoring*):

$$\begin{aligned}
C_{scaler} &= 16 \times 5 = 80 \text{ siklus} \\
C_{traversal} &= N_{pohon} \times d_{rata-rata} \times C_{per_node} = 50 \times 13,17 \times 15 \approx 9.878 \text{ siklus} \\
C_{overhead} &= N_{pohon} \times 10 = 50 \times 10 = 500 \text{ siklus} \\
C_{agregasi} &= N_{pohon} \times C_{agg/pohon} = 50 \times 6 = 300 \text{ siklus} \\
C_{argmax} &= 35 \text{ siklus} \\
C_{total} &= 80 + 9.878 + 500 + 300 + 35 = \mathbf{10.793} \text{ siklus}
\end{aligned}$$

HistGB memiliki kedalaman rata-rata traversal yang lebih tinggi ($d \approx 13,17$) dibanding GB ($d = 10$), sehingga komponen traversal per pohon lebih besar ($197,6 = 13,17 \times 15$ vs. $150 = 10 \times 15$ siklus/pohon). Namun, karena jumlah pohon sama (50), perbedaan total hanya $9.878 - 7.500 = 2.378$ siklus, menghasilkan selisih latensi $\approx 37 \mu s$.

2.4.3 Hasil Waktu Inferensi

Waktu inferensi dihitung dari jumlah siklus dibagi frekuensi clock 64 MHz:

$$t_{inferensi} = \frac{C_{total}}{f_{clock}} = \frac{C_{total}}{64 \times 10^6} \text{ detik} \quad (6)$$

Kedua model yang memenuhi syarat kualitas (GB dan HistGB) menunjukkan latensi di bawah $170 \mu s$. Gradient Boosting memiliki latensi terendah ($131,5 \mu s$) berkat kombinasi jumlah pohon yang sedikit (50) dan kedalaman rata-rata yang rendah (10). HistGradient Boosting sedikit lebih lambat

Table 10: Hasil Latensi Inferensi pada nRF52840 @ 64 MHz

Model	Siklus	Latensi (μ s)	Latensi (ms)	Throughput (inf/s)
Extra Trees [†]	16.500	257,8	0,258	3.878
Random Forest [†]	16.500	257,8	0,258	3.878
Gradient Boosting	8.415	131,5	0,132	7.607
HistGrad. Boosting	10.793	168,6	0,169	5.930

[†]Tidak memenuhi syarat kualitas ($\Delta F1_M < -0,01$); disertakan sebagai referensi.

(168,6 μ s) akibat kedalaman rata-rata pohon yang lebih tinggi (13,18 vs. 10,0), namun mengonsumsi hanya 6,4% memori *flash* dibanding GB.

2.4.4 Estimasi Konsumsi Energi

Konsumsi energi per inferensi dihitung berdasarkan karakteristik daya nRF52840:

$$E_{inferensi} = I_{aktif} \times V_{supply} \times t_{inferensi} \quad (7)$$

Dengan arus aktif 4,8 mA pada 64 MHz dan tegangan 3,0V (daya $P = 14,4$ mW):

Table 11: Konsumsi Energi per Inferensi pada nRF52840

Model	Latensi (μ s)	Daya (mW)	Energi (μ J)
Extra Trees [†]	257,8	14,4	3,71
Random Forest [†]	257,8	14,4	3,71
Gradient Boosting	131,5	14,4	1,89
HistGrad. Boosting	168,6	14,4	2,43

Sebagai perbandingan, transmisi radio IEEE 802.15.4 pada nRF52840 mengonsumsi sekitar 0,5–1 mJ per paket (termasuk overhead *Clear Channel Assessment* dan *acknowledgment*). Energi inferensi model terbesar yang layak (HistGradient Boosting, 2,43 μ J) hanya mewakili sekitar 0,2–0,5% dari energi transmisi radio — dapat diabaikan.

2.5 Analisis dan Pembahasan

2.5.1 Kesesuaian dengan Batasan Waktu Protokol LEACH

Dalam protokol LEACH, selama fase *steady-state* (≈ 18 –19 detik), CH menerima data dari anggota cluster dengan interval 1–2 detik per anggota. Dalam skenario terburuk dengan 10 paket/detik:

$$t_{paket} = 100 \text{ ms} \quad (8)$$

Rasio margin waktu untuk model yang layak dengan latensi terbesar (HistGradient Boosting):

$$\text{Margin} = \frac{t_{paket}}{t_{inferensi}} = \frac{100 \text{ ms}}{0,169 \text{ ms}} \approx 592\times \quad (9)$$

Margin waktu sebesar $592\times$ memastikan bahwa sistem IDS dapat beroperasi secara *real-time* tanpa mengganggu operasi normal protokol. Untuk model Gradient Boosting, margin mencapai $\approx 763\times$.

2.5.2 Perbandingan antar Model

1. **Extra Trees:** Model ini **tidak layak untuk deployment** karena degradasi kualitas yang parah ($\Delta F1_M = -0,140$). Masalah mendasarnya bukan pada keterbatasan platform, melainkan pada sifat intrinsik Extra Trees yang memerlukan pohon sangat dalam (kedalaman 20+) akibat pemilihan *split* acak. Dalam batasan memori 800 KB, kedalaman maksimum yang dapat dicapai adalah 10, yang menghilangkan sebagian besar kapasitas diskriminatif model, terutama pada kelas minoritas.
2. **Random Forest:** Model ini berada di **zona marginal** ($\Delta F1_M = -0,029$). Meskipun tidak memenuhi ambang batas ketat $-0,01$, degradasinya lebih moderat dibanding ET dan terkonsentrasi pada kelas Grayhole ($\Delta F1 = -0,114$). Untuk aplikasi yang dapat mentoleransi sedikit penurunan pada kelas minoritas, RF pada kedalaman 10 masih dapat dipertimbangkan.
3. **Gradient Boosting:** Model ini menunjukkan performa **sangat baik** ($\Delta F1_M = -0,005$). Temuan kunci: hanya 10 dari 100 *stage* boosting diperlukan, dan pohon secara alami memiliki kedalaman ≤ 10 sehingga **tidak ada pemotongan pohon**. Kualitas yang tinggi menunjukkan bahwa iterasi boosting awal sudah menangkap pola diskriminatif utama dalam data WSN. Kekurangannya adalah penggunaan memori yang relatif besar (571 KB, 55,8% flash) karena banyaknya node per pohon (rata-rata 973 node/pohon).
4. **Histogram Gradient Boosting:** Model ini merupakan **pilihan optimal** untuk deployment. Kualitas justru sedikit meningkat ($\Delta F1_M =$

+0,003) dengan penggunaan memori hanya 36,3 KB (3,6% flash). Keunggulan HistGB berasal dari representasi histogram yang menghasilkan pohon sangat ringkas (61 node/pohon) dibanding GB (973 node/pohon). Hanya 10 dari 66 iterasi diperlukan, dan kedalaman alami pohon (hingga 15) dipertahankan sepenuhnya. Model ini menyisakan lebih dari 960 KB flash untuk *protocol stack*, *driver*, dan kode aplikasi.

2.5.3 Superioritas Model Boosting untuk Deployment Terbatas Memori

Hasil evaluasi mengungkapkan temuan yang kontra-intuitif: model *boosting* lebih cocok untuk deployment pada mikrokontroler dibanding model *bagging*, meskipun model *bagging* secara tradisional dianggap lebih *robust* dan mudah diparalelkan. Dua faktor utama menjelaskan fenomena ini:

1. **Reduksi *stage* vs. pemotongan kedalaman:** Model *boosting* memungkinkan pengurangan jumlah *stage* (iterasi boosting) tanpa mengubah struktur pohon individual. Pada model *bagging*, satu-satunya opsi penghematan memori adalah pemotongan kedalaman, yang secara langsung mengurangi kapasitas diskriminatif setiap pohon.
2. **Konsentrasi informasi pada iterasi awal:** Dalam *gradient boosting*, iterasi pertama menangkap pola utama (sinyal kuat), sementara iterasi selanjutnya memperbaiki residual yang semakin kecil. Oleh karena itu, 10 *stage* pertama dari GB/HistGB sudah merepresentasikan lebih dari 97% kualitas model penuh. Sebaliknya, pada model *bagging*, setiap pohon berkontribusi secara relatif merata terhadap prediksi akhir, sehingga mengurangi jumlah pohon akan menurunkan kualitas secara proporsional.

2.5.4 Keunggulan Pendekatan Float32 Native

Penggunaan representasi *float32* native pada ARM Cortex-M4F memberikan beberapa keunggulan signifikan dibanding pendekatan kuantisasi INT8:

1. **Eliminasi *Error* Kuantisasi:** Tidak ada degradasi akurasi akibat konversi tipe data. Satu-satunya sumber penurunan akurasi adalah pembatasan kedalaman/*stage*, yang dapat dikontrol dan dievaluasi secara langsung.

2. **Penghematan Siklus Kuantisasi:** Tahap kuantisasi fitur (≈ 650 siklus pada MSP430) sepenuhnya dihilangkan. Overhead pra-pemrosesan *StandardScaler* (80 siklus) jauh lebih kecil.
3. **Evaluasi Kualitas yang Transparan:** Setiap perbedaan antara prediksi model asli dan versi *embedded* dapat diatribusikan secara langsung ke pembatasan kedalaman atau pengurangan *stage*, memudahkan analisis dan *debugging*.
4. **Konsistensi Numerik:** Hasil inferensi pada mikrokontroler dijamin identik dengan simulasi Python (kecuali pembulatan IEEE 754 yang minimal).

2.5.5 Implikasi untuk Deployment WSN

Hasil evaluasi mengkonfirmasi kelayakan deployment model IDS berbasis *tree ensemble* pada platform ARM Cortex-M4F:

1. **Efisiensi Memori:** Model HistGradient Boosting (pilihan optimal) hanya menggunakan 36,3 KB (3,6% flash), menyisakan lebih dari 960 KB untuk:
 - Stack protokol LEACH dan MAC 802.15.4 (≈ 50 –80 KB)
 - *Softdevice* Nordic (jika menggunakan BLE) (≈ 152 KB)
 - Driver radio dan HAL (≈ 40 KB)
 - Kode aplikasi sensor dan *buffer* (≈ 30 –50 KB)
2. **Kapabilitas *Real-time*:** Model yang layak menunjukkan latensi 132–169 μ s, memungkinkan inspeksi per-paket tanpa *buffering* dengan margin waktu $> 590\times$ terhadap batasan LEACH.
3. **Efisiensi Energi:** Konsumsi energi inferensi (1,89–2,43 μ J) sangat kecil dibanding energi transmisi radio ($\approx 0,5$ –1 mJ per paket), sehingga tidak berdampak signifikan pada masa pakai baterai.
4. **Kualitas Tinggi:** Dengan Conservative SMOTE, model asli mencapai akurasi $> 99\%$ dan $F1_{macro} > 96\%$. Model yang di-deploy mempertahankan kualitas ini ($|\Delta F1_M| < 0,01$) untuk model *boosting*, menghasilkan sistem IDS *on-device* yang akurat dan efisien.

2.5.6 Keterbatasan Evaluasi

Beberapa keterbatasan dari metodologi evaluasi ini perlu diperhatikan:

1. **Estimasi Siklus Statis:** Penghitungan siklus dilakukan berdasarkan analisis *disassembly* (15 siklus/node), bukan pengukuran langsung menggunakan DWT *Cycle Counter* pada perangkat keras fisik. Variasi akibat *flash wait-state*, *pipeline stall*, atau *interrupt* tidak diperhitungkan. Latensi instruksi *load* (VLDR, LDRH) sudah diperhitungkan sebesar 2 siklus sesuai TRM, namun pada kondisi tertentu (misalnya akses *flash* dengan *wait-state* tambahan), latensi aktual dapat sedikit lebih tinggi.
2. **Presisi *Float32* pada HistGB:** Evaluasi kualitas menggunakan data *float64* (presisi ganda) untuk memastikan konsistensi dengan hasil *scikit-learn*. Pada deployment nyata, data masukan akan dikonversi ke *float32*. Pengujian menunjukkan bahwa konversi ini berpotensi memengaruhi model HistGradient Boosting karena batas histogram yang sangat ketat, meskipun efeknya bervariasi tergantung distribusi data masukan. Validasi menggunakan data *float32* pada perangkat fisik direkomendasikan sebelum deployment produksi.
3. **Pembatasan Model *Bagging*:** Ketidaklayakan ET dan degradasi RF bukan disebabkan oleh keterbatasan platform ARM, melainkan oleh kebutuhan intrinsik model *bagging* akan pohon yang dalam. Alternatif seperti pengurangan jumlah pohon (dari 100) atau pelatihan ulang dengan kedalaman terbatas dapat menghasilkan model *bagging* yang lebih sesuai untuk deployment.
4. **Pengukuran Tunggal:** Evaluasi dilakukan pada satu konfigurasi data (*test set* tetap). Variasi latensi akibat distribusi data yang berbeda (kedalaman traversal bervariasi per sampel) dicakup oleh analisis kasus terbaik dan terburuk yang telah dilakukan.

2.6 Ringkasan Evaluasi ARM Cortex-M4F

Evaluasi latensi inferensi pada mikrokontroler nRF52840 (ARM Cortex-M4F @ 64 MHz) dengan model Conservative SMOTE menunjukkan bahwa:

1. Model *tree ensemble* dalam format *float32* native dapat dieksekusi dengan latensi 132–258 μ s tanpa kuantisasi, memanfaatkan FPU perangkat keras untuk perbandingan *floating-point* dalam 1 siklus. Penambahan pra-pemrosesan *StandardScaler* hanya menambah 80 siklus (<1% overhead).

Table 12: Ringkasan Evaluasi Deployment pada nRF52840 ARM Cortex-M4F (Conservative SMOTE)

Model	Pohon	Flash (KB)	Latensi (μs)	Energi (μJ)	$\Delta F1_M$	Layak
Extra Trees	100	581,1	257,8	3,71	-0,140	Tidak
Random Forest	100	618,6	257,8	3,71	-0,029	Marginal
Grad. Boosting	50	570,9	131,5	1,89	-0,005	Ya
HistGrad. Boosting	50	36,3	168,6	2,43	+0,003	Ya

2. **Dua dari empat model** — Gradient Boosting ($\Delta F1_M = -0,005$) dan Histogram Gradient Boosting ($\Delta F1_M = +0,003$) — memenuhi seluruh batasan deployment: memori (≤ 571 KB), latensi ($\leq 169 \mu s$), dan kualitas ($|\Delta F1_M| < 0,01$). Model *boosting* unggul karena pengurangan jumlah *stage* lebih efektif daripada pemotongan kedalaman pohon.
3. **Histogram Gradient Boosting** merupakan pilihan optimal dengan memori hanya 36,3 KB (3,6% flash), menyisakan ruang maksimal untuk *firmware* WSN. Kualitas model justru meningkat ($\Delta F1_M = +0,003$) dengan hanya 10 dari 66 iterasi, menunjukkan efisiensi representasi histogram yang luar biasa.
4. Model *bagging* (Extra Trees, Random Forest) **tidak optimal** untuk deployment pada batasan memori 800 KB karena memerlukan pohon yang sangat dalam. Extra Trees menunjukkan degradasi yang parah ($\Delta F1_M = -0,140$), sementara Random Forest marginal ($\Delta F1_M = -0,029$).
5. Throughput inferensi (3.878–7.607 inferensi/detik) melebihi kebutuhan tipikal WSN, dan konsumsi energi per inferensi (1,89–3,71 μJ) dapat diabaikan dibanding energi transmisi radio ($< 0,8\%$).